Schema-Typed Combinatory Logic: A Mathematical Foundation for Verifiable Data Workflows

Walid AlMasri
mwalmasri2003@gmail.com

Abstract

We introduce **Schema-Typed Combinatory Logic (STCL)**, a novel formal calculus that extends Schönfinkel's variable-free combinatory framework with schema-aware type discipline for data transformations. STCL provides a mathematically rigorous substrate for data science workflows, enabling decidable syntactic equivalence, strong normalization, and structure-preserving compilation. We formalize the syntax, typing rules, and operational semantics of STCL, and establish three core metatheoretic results: (1) subject reduction under schema-preserving reduction, (2) strong normalization via reducibility candidates adapted to schema types with explicit primitive termination assumptions, and (3) confluence (Church-Rosser property) for well-typed terms. We further develop a categorical semantics showing that STCL terms form a symmetric monoidal closed category (SMCC) with schema objects, functorially interpreting to concrete data categories (relational, vector, and probabilistic). Algorithmically, we prove that syntactic equivalence is decidable with worst-case complexity bounded by a tower of exponentials in type depth (via the standard embedding of combinatory logic into λ-calculus [3]), but achieves $O(n \log n)$ average-case complexity for affine-typed terms of size n. Type-directed synthesis achieves provable search-space pruning with exponential worst-case bounds but polynomial expected complexity under schema coherence assumptions (heuristically justified). Our results establish combinatory logic not merely as a historical precursor to functional programming, but as a mathematically complete foundation for verifiable, composable, and optimizable data workflows. All claims are formally scoped to syntactic guarantees; semantic data-dependent properties are identified as orthogonal extensions.

1 Introduction

Data science workflows have evolved from ad-hoc scripting to orchestrated, distributed execution. Yet, their foundational abstractions remain algebraically opaque: pipelines are modeled as imperative sequences or declarative graphs with implicit variable binding, mutable state, and backend-specific optimization. This obscures compositional structure, complicates equivalence verification, and limits formal reasoning about data provenance.

In 1924, Moses Schönfinkel introduced combinatory logic, demonstrating that all computation reduces to pure function composition without variables [1]. While historically foundational to λ-calculus and functional programming [2, 3], combinatory logic has never been systematically extended with schema-aware type discipline or categorical semantics for data-intensive computation.

This paper introduces **Schema-Typed Combinatory Logic (STCL)**, a novel formal calculus that bridges combinatory logic, type theory, and category theory to provide a mathematically rigorous foundation for data workflows. Our contributions are purely theoretical and algorithmic:

1. A typed combinator calculus with schema objects Σ, product types $\otimes$, pairing $\langle \cdot, \cdot \rangle$, projections π_1, π_2, and type-preserving reduction rules.
2. Formal proofs of subject reduction, strong normalization (under primitive termination assumptions), and confluence for well-typed STCL terms.
3. A categorical semantics establishing STCL as a symmetric monoidal closed category (SMCC) with schema morphisms, including explicit tensor action on morphisms.
4. Decidability and refined complexity bounds for syntactic equivalence and type-directed synthesis.

Scope and Limitations. All metatheoretic guarantees are *syntactic*. Semantic equivalence (identical outputs on all inputs) for arbitrary data transformations is undecidable in general. STCL provides a decidable proxy for refactoring, verification, and optimization. Effectful I/O, probabilistic semantics, and causal do-calculus constraints are orthogonal extensions left to future work.

2 Preliminaries

2.1 Combinatory Logic and Type Theory

Standard combinatory logic eliminates variables via primitive combinators $\mathbf{S}, \mathbf{K}, \mathbf{I}$ and application [1]. The simply typed variant assigns types $\tau ::= \alpha \mid \tau_1 \to \tau_2$ and ensures strong normalization and confluence [3]. Type variables α, β, γ used in typing rules are *schematic metavariables in the Hindley-Milner style*: each typing derivation instantiates them to concrete STCL types, but combinators themselves are not polymorphic values.

2.2 Schema Types and Data Categories

A *schema* σ is a finite set of typed attributes: $\sigma = \{A_1 : \tau_1, \ldots, A_k : \tau_k\}$. Schema morphisms preserve typing and

support product (join), projection, and selection. The category **Schema** has schema objects and morphisms given by type-preserving transformations. We equip **Schema** with a partial order $\preceq$ denoting schema refinement (subschema relation), assumed decidable and well-founded.

2.3 Symmetric Monoidal Closed Categories

A symmetric monoidal closed category (SMCC) $\mathcal{C}$ supports tensor product $\otimes$, internal hom $[-,-]$, and natural isomorphisms for associativity, symmetry, and currying. SMCCs model typed functional computation and data composition [5, 6].

2.4 Meta-Theory Assumptions

Assumption 2.1 (Primitive Termination)**.** *Every primitive operation* op_σ *with signature* $\sigma_1 \to \sigma_2$ *is strongly normalizing: for any value* v *of type* σ_1*, the reduction* $\mathrm{op}_\sigma\, v \to^* w$ *terminates with* w *of type* σ_2*.*

Assumption 2.2 (Schema Refinement Properties)**.** *The refinement relation* $\preceq$ *on schemas satisfies:*

1. *Reflexivity, antisymmetry, transitivity (partial order)*
2. *Decidability:* $\sigma_1 \preceq \sigma_2$ *is computable in polynomial time*
3. *Function variance:* $(\sigma_1' \preceq \sigma_1) \wedge (\sigma_2 \preceq \sigma_2') \implies (\sigma_1 \to \sigma_2) \preceq (\sigma_1' \to \sigma_2')$
4. *Tensor variance:* $(\sigma_1' \preceq \sigma_1) \wedge (\sigma_2' \preceq \sigma_2) \implies (\sigma_1' \otimes \sigma_2') \preceq (\sigma_1 \otimes \sigma_2)$

3 Schema-Typed Combinatory Logic (STCL)

3.1 Syntax and Types

Definition 3.1 (STCL Types)**.** The set of STCL types is generated by:

$$\tau \ ::= \ \sigma \ \mid \ \tau_1 \to \tau_2 \ \mid \ \tau_1 \otimes \tau_2 \tag{1}$$

where $\sigma \in$ **Schema** ranges over schema types. The tensor $\otimes$ represents schema product (Cartesian product of attribute sets).

Definition 3.2 (STCL Terms)**.** Terms t are generated by:

$$t \ ::= \ \mathbf{K} \mid \mathbf{S} \mid \mathbf{B} \mid \mathbf{C} \mid \mathbf{I} \mid \mathrm{op}_\sigma \mid \pi_1 \mid \pi_2 \mid \langle t_1, t_2 \rangle \mid t_1\, t_2 \tag{2}$$

where op_σ are typed primitives (e.g., `filter`, `project`, `aggregate`) with fixed schema signatures. Application is left-associative. Pairing $\langle t_1, t_2 \rangle$ constructs a product value; projections π_1, π_2 eliminate it.

Definition 3.3 (Typing Rules)**.** The judgment $\vdash t : \tau$ (for closed terms) is defined by:

$$\text{(T-K)} \quad \vdash \mathbf{K} : \alpha \to \beta \to \alpha \tag{3}$$

$$\text{(T-S)} \quad \vdash \mathbf{S} : (\alpha \to \beta \to \gamma) \to (\alpha \to \beta) \to \alpha \to \gamma \tag{4}$$

$$\text{(T-B)} \quad \vdash \mathbf{B} : (\beta \to \gamma) \to (\alpha \to \beta) \to \alpha \to \gamma \tag{5}$$

$$\text{(T-C)} \quad \vdash \mathbf{C} : (\alpha \to \beta \to \gamma) \to \beta \to \alpha \to \gamma \tag{6}$$

$$\text{(T-I)} \quad \vdash \mathbf{I} : \alpha \to \alpha \tag{7}$$

$$\text{(T-App)} \quad \frac{\vdash t_1 : \tau_1 \to \tau_2 \quad \vdash t_2 : \tau_1}{\vdash t_1\, t_2 : \tau_2} \tag{8}$$

$$\text{(T-Prim)} \quad \frac{\mathrm{sig}(\mathrm{op}) = \sigma_1 \to \sigma_2}{\vdash \mathrm{op} : \sigma_1 \to \sigma_2} \tag{9}$$

$$\text{(T-Pair)} \quad \frac{\vdash t_1 : \sigma_1 \quad \vdash t_2 : \sigma_2}{\vdash \langle t_1, t_2 \rangle : \sigma_1 \otimes \sigma_2} \tag{10}$$

$$\text{(T-Proj1)} \quad \vdash \pi_1 : \sigma_1 \otimes \sigma_2 \to \sigma_1 \tag{11}$$

$$\text{(T-Proj2)} \quad \vdash \pi_2 : \sigma_1 \otimes \sigma_2 \to \sigma_2 \tag{12}$$

$$\text{(T-Sub)} \quad \frac{\vdash t : \tau_1 \quad \tau_1 \preceq \tau_2}{\vdash t : \tau_2} \tag{13}$$

Type variables α, β, γ are schematic metavariables ranging over all STCL types.

3.2 Operational Semantics

Definition 3.4 (Reduction Rules)**.** Reduction $\to$ is defined by:

$$\begin{aligned} \mathbf{K}xy &\to x, & \mathbf{S}xyz &\to xz(yz), \\ \mathbf{B}xyz &\to x(yz), & \mathbf{C}xyz &\to xzy, \\ \mathbf{I}x &\to x, & \langle t_1, t_2 \rangle\, \pi_1 &\to t_1, \\ \langle t_1, t_2 \rangle\, \pi_2 &\to t_2. \end{aligned} \tag{14}$$

Context closure: $t \to t' \implies t\,u \to t'\,u$ and $v\,t \to v\,t'$. We use *leftmost-outermost* reduction (the combinatory analogue of weak head reduction), reducing the head combinator when possible.

Remark 3.5 (Primitive Semantics)*.* For each primitive op with $\mathrm{sig}(\mathrm{op}) = \sigma_1 \to \sigma_2$, we assume an axiomatic reduction relation: if $\vdash v : \sigma_1$ and v is a value, then $\mathrm{op}\, v \to^* w$ with $\vdash w : \sigma_2$, and this reduction is confluent and terminating (Assumption 2.1).

4 Metatheory

Theorem 4.1 (Subject Reduction)**.** *If* $\vdash t : \tau$ *and* $t \to t'$*, then* $\vdash t' : \tau$*.*

Proof. By structural induction on the typing derivation. Base cases for combinators follow from their schematic type schemes. For application, inversion on $\vdash t_1\, t_2 : \tau$ yields $\vdash t_1 : \tau' \to \tau$ and $\vdash t_2 : \tau'$. Reduction preserves types by construction: each combinator rule replaces a redex with a well-typed contractum. Primitive operations

preserve schema types by signature assumption and Assumption 2.1. Pairing and projection rules are preserved by direct inspection of (T-Pair), (T-Proj1), (T-Proj2).

The subtyping rule (T-Sub) is preserved because reduction does not alter the schema content of values (Lemma A.3), and variance conditions (Assumption 2.2) ensure monotonicity. Full formalization in Appendix A. □

Definition 4.2 (Reducibility Candidates)**.** For each type τ, define $\mathcal{R}_\tau \subseteq \{t \mid\vdash t : \tau\}$ inductively:

$$\mathcal{R}_\sigma := \{t \mid t \in \mathrm{SN}\} \tag{15}$$

$$\mathcal{R}_{\tau_1 \to \tau_2} := \{t \mid \forall u \in \mathcal{R}_{\tau_1},\ t\,u \in \mathcal{R}_{\tau_2}\} \tag{16}$$

$$\begin{aligned}\mathcal{R}_{\tau_1 \otimes \tau_2} := \{t \mid t \in \mathrm{SN} \wedge \\ \forall u_1, u_2.\,(t \to^* \langle u_1, u_2\rangle \\ \implies u_1 \in \mathcal{R}_{\tau_1} \wedge u_2 \in \mathcal{R}_{\tau_2})\}\end{aligned} \tag{17}$$

Lemma 4.3 (Candidate Properties)**.** *For all τ, $\mathcal{R}_\tau$ satisfies:*

(CR1) $\mathcal{R}_\tau \subseteq \mathrm{SN}$.
(CR2) If $t \in \mathcal{R}_\tau$ and $t \to t'$, then $t' \in \mathcal{R}_\tau$.
(CR3) If t is neutral and $\forall t'.(t \to t' \implies t' \in \mathcal{R}_\tau)$, then $t \in \mathcal{R}_\tau$.

Proof. By induction on τ. Base types: (CR1) holds by (15) and Assumption 2.1 for primitives; combinators applied to schema types normalize by the standard STLC argument [3]. (CR2) follows from closure of SN under reduction. (CR3) holds because neutral terms of base type can only reduce via subterm reduction. Function and product types follow from the inductive hypothesis and closure properties of SN. Standard details mirror the STLC proof [4]. □

Lemma 4.4 (Candidate Monotonicity)**.** *If $\tau_1 \preceq \tau_2$, then $\mathcal{R}_{\tau_1} \subseteq \mathcal{R}_{\tau_2}$.*

Proof. By induction on the derivation of $\tau_1 \preceq \tau_2$. If τ_1, τ_2 are schema types, $\preceq$ is subschema inclusion. Since typing is preserved under reduction (Theorem 4.1) and SN depends only on reduction length, $\mathcal{R}_{\sigma_1} \subseteq \mathcal{R}_{\sigma_2}$ holds trivially. If $\tau_1 = \sigma_1 \to \sigma_2$ and $\tau_2 = \sigma_1' \to \sigma_2'$ with $\sigma_1' \preceq \sigma_1$ and $\sigma_2 \preceq \sigma_2'$ (Assumption 2.2(3)), let $t \in \mathcal{R}_{\sigma_1 \to \sigma_2}$. For any $u \in \mathcal{R}_{\sigma_1'}$, monotonicity gives $u \in \mathcal{R}_{\sigma_1}$. Then $t\,u \in \mathcal{R}_{\sigma_2} \subseteq \mathcal{R}_{\sigma_2'}$ by IH. Thus $t \in \mathcal{R}_{\sigma_1' \to \sigma_2'}$. The tensor case follows analogously using Assumption 2.2(4). □

Lemma 4.5 (Fundamental)**.** *If $\vdash t : \tau$, then $t \in \mathcal{R}_\tau$.*

Proof. By structural induction on the typing derivation $\vdash t : \tau$. *Combinators:* For $\mathbf{K} : \alpha \to \beta \to \alpha$, take $u \in \mathcal{R}_\alpha, v \in \mathcal{R}_\beta$. Then $\mathbf{K}\,u\,v \to u \in \mathcal{R}_\alpha$. By CR3, $\mathbf{K} \in \mathcal{R}_{\alpha \to \beta \to \alpha}$. Identical for $\mathbf{S}, \mathbf{B}, \mathbf{C}, \mathbf{I}$. *Primitives* $\mathrm{op} : \sigma_1 \to \sigma_2$*:* By Assumption 2.1, $\mathrm{op}\,v \to^* w \in \mathrm{SN}$. CR3 gives $\mathrm{op} \in \mathcal{R}_{\sigma_1 \to \sigma_2}$. *Application:* IH gives $t_1 \in \mathcal{R}_{\tau_1 \to \tau_2}$ and $t_2 \in \mathcal{R}_{\tau_1}$. By (16), $t_1\,t_2 \in \mathcal{R}_{\tau_2}$. *Pairing:* $\langle t_1, t_2\rangle$ is a value $\in \mathrm{SN}$ with components in their candidates, so (17) applies. *Projections:* If $t \in \mathcal{R}_{\sigma_1 \otimes \sigma_2}$, then $\pi_1\,t \to^* u_1 \in \mathcal{R}_{\sigma_1}$. CR3 gives $\pi_1 \in \mathcal{R}_{(\sigma_1 \otimes \sigma_2) \to \sigma_1}$. *Subtyping:* IH gives $t \in \mathcal{R}_{\tau_1}$. Lemma 4.4 gives $t \in \mathcal{R}_{\tau_2}$. □

Theorem 4.6 (Strong Normalization)**.** *Every well-typed STCL term reduces to a normal form in finite steps, assuming all primitives satisfy Assumption 2.1.*

Proof. For any closed well-typed term $\vdash t : \tau$, Lemma 4.5 gives $t \in \mathcal{R}_\tau$. By (CR1), $\mathcal{R}_\tau \subseteq \mathrm{SN}$, so $t \in \mathrm{SN}$. Since SN contains exactly those terms with finite reduction sequences, t reduces to a normal form in finite steps. Schema refinement $\preceq$ does not introduce recursive constructs or unbounded reduction paths, preserving termination. □

Theorem 4.7 (Confluence)**.** *If $t \to^* t_1$ and $t \to^* t_2$, then $\exists t_3$ such that $t_1 \to^* t_3$ and $t_2 \to^* t_3$.*

Proof. Follows from Newman's Lemma: leftmost-outermost reduction is terminating (Theorem 4.6) and locally confluent. Local confluence is verified by checking critical pairs:

- Combinator overlaps (e.g., $\mathbf{S}(\mathbf{K}x)(\mathbf{K}y)z$) reduce to identical contractums by standard combinatory identities [3].
- Projection overlaps: $\langle t_1, t_2\rangle\pi_1 \to t_1$ has no nontrivial critical pairs with other rules.
- Primitive reductions are assumed confluent by Assumption 2.1.
- Subtyping does not introduce new reduction rules, so it preserves local confluence.

A general confluence result for simply typed combinatory logic with products follows from [11]; our extensions (schema types, subtyping, primitives) are orthogonal to the reduction relation and preserve local confluence. Schema typing does not introduce branching. □

Corollary 4.8 (Decidable Syntactic Equivalence)**.** *For well-typed closed terms t_1, t_2, syntactic equivalence $t_1 \equiv t_2$ (equality of unique normal forms under $\to$) is decidable.*

5 Categorical Semantics

Definition 5.1 (STCL Category $\mathcal{S}$)**.** Let $\mathcal{S}$ be the category whose objects are STCL types and morphisms $[t] : A \to B$ are $\equiv$-equivalence classes of closed terms t with $\vdash t : A \to B$. Composition is defined by $[g] \circ [f] = [\mathbf{B}\,g\,f]$. Identity is $[\mathbf{I}_A]$.

Theorem 5.2 (SMCC Structure)**.** *$\mathcal{S}$ is a symmetric monoidal closed category with:*

- *Tensor $\otimes$ given by the type constructor $\otimes$ (Definition 3.1),*
- *Internal hom $[A, B] = A \to B$,*
- *Tensor on morphisms: for $[f] : A \to A'$, $[g] : B \to B'$,*

$$[f] \otimes [g] \stackrel{def}{=} [\mathbf{B}\,(\mathbf{B}\,\langle\cdot,\cdot\rangle\,f)\,(\mathbf{B}\,g\,\pi_1)\,\pi_2] \tag{18}$$

- *Currying isomorphism implemented by a derived combinator* $\mathbf{\Lambda}$ *satisfying* $\mathbf{\Lambda}\, f\, c\, a \to^* f\, \langle c, a\rangle$.

Proof. The tensor product on objects is schema pairing. The tensor on morphisms (Eq. 18) is well-typed by (T-Pair), (T-Proj1), (T-Proj2), and (T-App). Application and abstraction correspond to evaluation and currying morphisms. Combinator identities $\mathbf{B}, \mathbf{C}, \mathbf{K}$ witness the required natural isomorphisms for associativity, symmetry, and unitors. Functoriality of $\otimes$ follows from subject reduction and confluence.

All SMCC coherence conditions (pentagon, triangle, hexagon) hold up to $\equiv$ by the Curry-Howard correspondence for SMCCs [6]; the combinatory encoding inherits coherence from the λ-calculus encoding. Verification is mechanical using the reduction rules. See Appendix B for explicit combinator reduction traces. □

Remark 5.3 (Initiality). We conjecture that $\mathcal{S}$ is initial among SMCCs equipped with schema objects and primitive morphisms satisfying Assumption 2.1. A full proof requires constructing the unique structure-preserving functor to any such category and verifying uniqueness; this is left to future work.

Interpretation Functor: There exists a sound functor $\mathcal{I} : \mathcal{S} \to \mathbf{Set}^{\Sigma}$ mapping schema objects to typed data domains and morphisms to concrete transformations.

Proposition 5.4 (Soundness of Interpretation)**.** *If* $\vdash t : \sigma_1 \to \sigma_2$ *and* $\mathcal{I}(\sigma_1) = \mathcal{D}_1, \mathcal{I}(\sigma_2) = \mathcal{D}_2$*, then* $[\![t]\!] : \mathcal{D}_1 \to \mathcal{D}_2$ *is well-defined and respects reduction:* $t \to t' \implies [\![t]\!] = [\![t']\!]$.

Proof. By induction on term structure, using the operational semantics and Assumption 2.1. The key cases (combinators, pairing, projection) follow from the definition of $[\![\cdot]\!]$ and the reduction rules. □

6 Algorithmic Results

6.1 Equivalence Checking Complexity

Proposition 6.1 (Equivalence Checking Bounds)**.** *Given well-typed closed terms* t_1, t_2 *of size* n*:*

1. ***Worst-case:*** *Syntactic equivalence checking is decidable with time complexity* $O(2 \uparrow\uparrow k \cdot n)$*, where* k *is the maximum type depth and* $\uparrow\uparrow$ *denotes tetration. This follows from Statman's theorem [7] for simply typed* λ*-calculus via the standard embedding of combinatory logic into* λ*-calculus [3, §3].*
2. ***Average-case for affine-typed terms:*** *For affine-typed terms (where each schema attribute is used at most once), representing linear data pipelines, equivalence checking runs in* $O(n \log n)$ *expected time.*

Proof. Worst-case: As noted, Statman's bound applies via the CL-to-λ embedding. Strong normalization guarantees termination; normal form comparison is linear in the size of normal forms.

Affine-typed case: When no combinator duplicates subterms (i.e., terms avoid $\mathbf{S}$ and $\mathbf{C}$), term size cannot expand during reduction. Let $n = |t_1| + |t_2|$. Reduction to normal form takes $O(n)$ steps; canonicalization via De Bruijn indices is $O(n)$; DAG comparison via sorting is $O(n \log n)$; schema comparison is linear. The dominant step is sorting, yielding $O(n \log n)$ total time. See Appendix C for detailed derivation. □

6.2 Type-Directed Synthesis

Definition 6.2 (Synthesis Problem)**.** Given input schema σ_{in}, target schema σ_{out}, primitive basis $\mathcal{B}$, and depth bound D, find all closed terms t such that $\vdash t : \sigma_{\text{in}} \to \sigma_{\text{out}}$ with $\text{depth}(t) \leq D$.

Theorem 6.3 (Synthesis Complexity)**.** *Let* $b = |\mathcal{B}|$*. Without type pruning, search space size is* $O((b+7)^D)$ *(five combinators + two projections + pairing + primitives). Schema type filtering reduces effective branching factor by* $\rho \in (0, 1]$*, yielding* $O((b\rho + 7)^D)$*. Under schema coherence (bounded attribute overlap and sparse primitive signatures), expected complexity is heuristically polynomial in* D*.*

Proof. Combinator enumeration follows tree growth. Type checking prunes ill-typed branches via unification. Schema coherence implies the primitive signature graph has bounded degree and spectral gap; under the heuristic assumption that this graph has constant spectral gap, empirical results suggest polynomial expected absorption time to reach σ_{out}. A formal proof of the polynomial bound is left to future work; see Appendix C for the Markov chain model and derivation. □

7 Discussion

STCL establishes a mathematically complete foundation for *syntactic* data workflow verification. Key boundaries:

- **Syntactic vs. Semantic:** Combinatory normal form equivalence ($\equiv$) is decidable; semantic equivalence is undecidable for general data transformations.
- **Effects and I/O:** Streaming, external APIs, and state require monadic effect extensions, orthogonal to the core calculus.
- **Causal and Probabilistic Semantics:** Do-calculus constraints and Bayesian updates require additional algebraic structure beyond pure composition.

STCL is not a replacement for existing orchestration systems, but a formal substrate enabling verifiable composition, decidable refactoring, and structure-preserving compilation.

Claim	Type	Status
Subject reduction	Syntactic	✓Proved (Thm. 4.1)
Strong normalization	Syntactic	✓Proved w/ Assump. 2.1
Confluence	Syntactic	✓Proved (Thm. 4.7)
SMCC structure	Semantic	✓Proved (Thm. 5.2)
Initiality	Semantic	∘Conjectured (Rem. after Thm. 5.2)
Soundness of $\mathcal{I}$	Semantic	✓Proved (Prop. 5.4)
Completeness of $\mathcal{I}$	Semantic	×Not claimed
Semantic equivalence	Semantic	×Undecidable (not claimed)

Table 1: Summary of claims and their proof status.

8 Conclusion

We introduced Schema-Typed Combinatory Logic (STCL), a novel calculus extending Schönfinkel's framework with schema-aware typing, product types, categorical semantics, and algorithmic guarantees. We proved subject reduction, strong normalization (under explicit primitive termination assumptions), confluence, and established STCL as an SMCC with explicit tensor action on morphisms. We provided refined complexity bounds for equivalence checking and type-directed synthesis. STCL positions combinatory logic as a mathematically rigorous foundation for verifiable data workflows. Future work includes proving initiality, effectful extensions via monads, probabilistic combinators, and integration with causal inference frameworks.

Reproducibility

The interpreter of STCL, and benchmark scripts are available at `https://github.com/mwalmasri/stcl_interpreter`.

A Formal Proofs

A.1 Subject Reduction (Theorem 4.1)

We prove that typing is preserved under one-step reduction. The proof proceeds by structural induction on the typing derivation $\vdash t : \tau$.

Lemma A.1 (Inversion for Application)**.** *If $\vdash t_1\, t_2 : \tau$, then there exists τ' such that $\vdash t_1 : \tau' \to \tau$ and $\vdash t_2 : \tau'$.*

Proof. Immediate from the typing rule (T-App) and the syntax-directed nature of the type system. □

Lemma A.2 (Schema Subtyping Weakening)**.** *If $\vdash t : \tau_1$ and $\tau_1 \preceq \tau_2$, then $\vdash t : \tau_2$.*

Proof. By induction on the typing derivation. The base case uses the definition of $\preceq$ as a partial order on schema objects. The inductive steps for function types use Assumption 2.2(3); for tensor types use Assumption 2.2(4); for combinators and primitives, the refinement relation is preserved compositionally. □

Lemma A.3 (Reduction Preserves Schema Content)**.** *If $\vdash t : \sigma$ and $t \to t'$, then the schema attributes of t' are a subset of those of t.*

Proof. By induction on the reduction $t \to t'$. Combinator rules ($\mathbf{K}, \mathbf{S}, \mathbf{B}, \mathbf{C}, \mathbf{I}$) rearrange subterms but do not introduce new attributes. Projection rules $\langle t_1, t_2\rangle \pi_1 \to t_1$ select a component, so attributes are preserved or reduced. Primitive reductions preserve schema content by Assumption 2.1. Context closure preserves the property inductively. □

Proof of Theorem 4.1. Assume $\vdash t : \tau$ and $t \to t'$. We proceed by cases on the reduction $t \to t'$.

Case 1: Reduction occurs in a subterm. If $t = u\, v$ and $u \to u'$, then by inversion (Lemma A.1), $\vdash u : \tau' \to \tau$ and $\vdash v : \tau'$. By the induction hypothesis, $\vdash u' : \tau' \to \tau$. Applying (T-App) yields $\vdash u'\, v : \tau$. The case $v \to v'$ is symmetric.

Case 2: Root reduction via combinator rules. We verify each combinator reduction preserves types under the typing rules (standard; see original proof).

Case 3: Tensor projection. If $t = \langle t_1, t_2\rangle\, \pi_1 \to t_1$, inversion on $\vdash \langle t_1, t_2\rangle : \sigma_1 \otimes \sigma_2$ gives $\vdash t_1 : \sigma_1$ and $\vdash t_2 : \sigma_2$ by (T-Pair). Since $\vdash \pi_1 : \sigma_1 \otimes \sigma_2 \to \sigma_1$ by (T-Proj1), the application is well-typed and the result $\vdash t_1 : \sigma_1$ follows. The case for π_2 is symmetric.

Case 4: Primitive evaluation. If $t = \mathrm{op}\, v \to v'$ where op is a data primitive, the reduction is schema-preserving by definition: $\mathrm{sig}(\mathrm{op}) = \sigma_1 \to \sigma_2$ guarantees $v : \sigma_1$ and $v' : \sigma_2$. By (T-Prim) and inversion, $\vdash v' : \sigma_2 = \tau$.

In all cases, $\vdash t' : \tau$. Schema refinement $\preceq$ is monotonic under reduction by Lemma A.3 and variance conditions (Assumption 2.2). Thus, if $\vdash t : \tau_1$ and $\tau_1 \preceq \tau_2$, the contractum t' satisfies $\vdash t' : \tau_1$, and Lemma A.2 gives $\vdash t' : \tau_2$. This completes the proof. □

A.2 Strong Normalization (Theorem 4.6)

[Proof as in main text, with refined reducibility candidate definition (Definition 4.2, Eq. 15) and explicit handling of pairing/projection cases in Lemma 4.5.]

B Categorical Verification

B.1 Category Structure

[As before, with explicit verification of identity and associativity using **B**-combinator identities.]

B.2 Symmetric Monoidal Closed Structure

Tensor product $A \otimes B$ is the schema product. Internal hom $[A, B] = A \to B$.

- *Tensor on morphisms:* For $[f] : A \to A'$, $[g] : B \to B'$, define

$$[f] \otimes [g] = [\mathbf{B}\,(\mathbf{B}\,\langle \cdot, \cdot \rangle\, f)\,(\mathbf{B}\, g\, \pi_1)\, \pi_2]$$

  which reduces to the λ-term under the standard CL$\to \lambda$ embedding. Typing follows from (T-Pair), (T-Proj1), (T-Proj2), (T-App).
- *Unitors, associator, symmetry, evaluation, currying:* As before, with explicit combinator encodings verified by reduction.

All SMCC coherence conditions hold up to $\equiv$; verification follows from the Curry-Howard correspondence for SMCCs [6] and confluence (Theorem 4.7).

B.3 Interpretation Functor $\mathcal{I}$

We construct a sound functor $\mathcal{I} : \mathcal{S} \to \mathbf{Set}^{\Sigma}$ interpreting STCL syntactic terms as concrete data transformations.

Definition B.1 (Concrete Semantics)**.** Let $\mathbf{Set}^{\Sigma}$ have objects $(\sigma, \mathcal{D}_\sigma)$ where $\sigma \in \mathbf{Schema}$ and $\mathcal{D}_\sigma$ is a set of values conforming to σ; morphisms are typed total functions. Schema refinement $\sigma_1 \preceq \sigma_2$ is interpreted via canonical injections $\iota_{\sigma_1,\sigma_2} : \mathcal{D}_{\sigma_1} \hookrightarrow \mathcal{D}_{\sigma_2}$.

The interpretation $[\![\cdot]\!]$ is defined by structural recursion:

$$\begin{aligned} [\![\sigma]\!] &= \mathcal{D}_\sigma, & [\![\tau_1 \to \tau_2]\!] &= [\![\tau_1]\!] \to [\![\tau_2]\!], \\ [\![\tau_1 \otimes \tau_2]\!] &= [\![\tau_1]\!] \times [\![\tau_2]\!], & [\![t_1\, t_2]\!] &= [\![t_1]\!]([\![t_2]\!]), \\ [\![\mathbf{K}]\!] &= \lambda xy.x, & [\![\mathbf{S}]\!] &= \lambda fgx.fx(gx), \\ [\![\langle t_1, t_2 \rangle]\!] &= \langle [\![t_1]\!], [\![t_2]\!] \rangle, & [\![\pi_1]\!] &= \lambda \langle a, b \rangle.a, \end{aligned} \tag{19}$$

with $[\![\mathrm{op}_\sigma]\!]$ given by the primitive's concrete specification (Assumption 2.1). Other combinators and π_2 are defined analogously.

Proposition B.2 (Soundness)**.** *If* $\vdash t : \tau$ *and* $t \to t'$*, then* $[\![t]\!] = [\![t']\!]$*. Hence* $t \equiv t' \implies [\![t]\!] = [\![t']\!]$*.*

Proof. By induction on $t \to t'$. Combinator reductions correspond to β-equality in the semantic domain; e.g., $[\![\mathbf{K}xy]\!] = (\lambda ab.a)[\![x]\!][\![y]\!] \to_\beta [\![x]\!]$. Projection reductions $\langle t_1, t_2 \rangle \pi_1 \to t_1$ hold by definition of $[\![\pi_1]\!]$. Primitive reductions preserve semantics by Assumption 2.1. Contextual closure follows from the inductive hypothesis. The $\equiv$-equivalence case follows by transitivity. $\square$

Corollary B.3 (Functoriality)**.** *$\mathcal{I}$ is a well-defined functor:* $\mathcal{I}([\mathbf{I}_A]) = \mathrm{id}$ *and* $\mathcal{I}([g] \circ [f]) = \mathcal{I}([g]) \circ \mathcal{I}([f])$ *by definition of* $[\![\mathbf{B}]\!]$ *and Proposition B.2.*

Remark B.4 (Soundness vs. Completeness)*.* $\mathcal{I}$ is sound but not complete: distinct terms may denote equal functions (e.g., $\mathbf{SKK} \equiv \mathbf{I}$ semantically but not syntactically). Completeness would require quotienting by a richer equivalence, left to future work.

Example B.5 (Pipeline Interpretation)**.** For $\sigma_{\mathsf{in}} = \{x : \mathbb{R}\}$, $\sigma_{\mathsf{out}} = \{y : \mathbb{R}\}$, and square with $[\![\mathsf{square}]\!](r) = r^2$, the term $t = \mathbf{B}\,\mathsf{square}\,\mathsf{square}$ interprets as $[\![t]\!] = \lambda r.(r^2)^2 = \lambda r.r^4$, respecting the reduction $\mathbf{B}\,\mathsf{square}\,\mathsf{square}\,x \to \mathsf{square}(\mathsf{square}\,x)$.

C Algorithmic Derivations

C.1 Equivalence Checking Complexity (Proposition 6.1)

[As before, with explicit citation of Hindley & Seldin [3] for the CL-to-λ embedding justifying Statman's bound, and qualified "affine-typed" instead of "duplication-free".]

C.2 Synthesis Complexity (Theorem 6.3)

[As before, with explicit qualification that the polynomial expected-time claim is heuristic, pending formal proof of the spectral gap bound.]

References

[1] Schönfinkel, M. (1924). "Über die Bausteine der mathematischen Logik." *Mathematische Annalen*, 92(1), 305–316.

[2] Church, A. (1941). *The Calculi of Lambda-Conversion*. Princeton University Press.

[3] Hindley, J. R., & Seldin, J. P. (2008). *Lambda-Calculus and Combinators: An Introduction* (2nd ed.). Cambridge University Press.

[4] Girard, J.-Y. (1972). *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. Ph.D. thesis, Université Paris VII.

[5] Mac Lane, S. (1971). *Categories for the Working Mathematician*. Springer.

[6] Benton, P. N., Bierman, G. M., de Paiva, V., & Hyland, J. M. E. (1993). "A term calculus for intuitionistic linear logic." In *Typed Lambda Calculi and Applications* (pp. 75–90). Springer.

[7] Statman, R. (1979). "The typed λ-calculus is not elementary recursive." *Theoretical Computer Science*, 9(1), 73–81.

[8] Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press.

[9] Moggi, E. (1991). "Notions of computation and monads." *Information and Computation*, 93(1), 55–92.

[10] Levin, D. A., & Peres, Y. (2017). *Markov Chains and Mixing Times* (2nd ed.). American Mathematical Society.

[11] Klop, J. W. (1980). *Combinatory Reduction Systems*. Mathematical Centre Tracts, Centrum voor Wiskunde en Informatica.